

Assignment Title:
System Internals

Coursework Type:
Individual

Module Name:
ST5003CMD – Operating Systems, Security, And Networks

Submitted By:
Name: Pranisha Bhatta
College ID: 250150

Module Leader:
Name: Rupak Rajbanshi

Table of Content:

I. Lab 1: Data Type Size Exploration: 2
A. C Source Code: 2
B. Terminal Execution & Verification: 3
C. Comparison with Table I: 3
D. Question: Why might sizeof(long) differ between 32-bit and 64-bit systems? 4
II. Lab 2: Memory Segment Address Mapping 5
A. C Source Code 5
B. Terminal Execution & Verification 6
C. Highest and Lowest Addresses 6
D. Calculating the Stack and Heap Address Difference 6
E. Stack vs Heap Comparison 6
III. Practical Code Example: Pointer and Memory Allocation: 7
A. C Source Code 7
B. Terminal Execution & Verification 8
C. Observation 8
IV. Summary and Key Takeaways 9
V. Conclusion: 10

Introduction

A data type is a contract between the programmer, the compiler and the operating system. It defines how many bytes are allocated for a value, how those bytes are interpreted, and which operations are valid on them. In this lab, we use C programs in the Ubuntu terminal to observe the size of different data types on our system and to see where the OS places variables in a process's memory.

I. Lab 1: Data Type Size Exploration:

Objective: Use sizeof() to observe how much memory each data type uses on this system.

A. C Source Code:

```
pranisha@ubuntu:~/Desktop$ touch lab1_sizes.c
```

```
pranisha@ubuntu:~/Desktop$ nano lab1_sizes.c
```

The touch command was used to create an empty file named lab1_sizes.c, and the nano editor was then opened to write the source code.

```
  GNU nano 8.7.1                                        lab1_sizes.c *
#include <stdio.h>

int main() {
        printf("=== Data Type Sizes on This System ===\n\n");
        printf("char:           %zu bytes\n", sizeof(char));
        printf("int:            %zu bytes\n", sizeof(int));
        printf("float:          %zu bytes\n", sizeof(float));
        printf("double:         %zu bytes\n", sizeof(double));
        printf("long:           %zu bytes\n", sizeof(long));
        printf("pointer:        %zu bytes\n", sizeof(int*));
        return 0;
}
```

The program uses the sizeof() operator to print the size in bytes of char, int, float, double, long, unsigned int and long long.

B. Terminal Execution & Verification:

```
pranisha@ubuntu:~/Desktop$ gcc lab1_sizes.c -o lab1
```

The program was compiled with gcc and no errors or warnings were shown, so the compilation was successful.

```
pranisha@ubuntu:~/Desktop$ ./lab1
=== Data Type Sizes on This System ===

char:           1 bytes
int:            4 bytes
float:          4 bytes
double:         8 bytes
long:           8 bytes
pointer:        8 bytes
```

Running the program printed the size of each data type on this system: char = 1 byte, int = 4 bytes, float = 4 bytes, double = 8 bytes, long = 8 bytes, unsigned int = 4 bytes and long long = 8 bytes.

C. Comparison with Table I:

Data Type	My Output (64-bit)	Table I(64-bit)	Match?
char	1 byte	1 byte	Yes
int	4 bytes	4 bytes	Yes
float	4 bytes	4 bytes	Yes
double	8 bytes	8 bytes	Yes
long	8 bytes	8 bytes	Yes
Unsigned int	4 bytes	not listed	Same as int
long long	8 bytes	8 bytes	Yes

The output was compared with the 64-bit column of Table I, and all values match. unsigned int is not listed in Table I, but it is 4 bytes, the same as int, since signed and unsigned versions of a type use the same amount of memory. The only value that would change on a 32-bit system is long.

D. Question: Why might sizeof(long) differ between 32-bit and 64-bit systems?

On a 32-bit system, long is 4 bytes, while on a 64-bit Linux system it is 8 bytes. This is because the size of long follows the architecture's word size, so on a 64-bit system it is large enough to hold a full 64-bit address or value.

II. Lab 2: Memory Segment Address Mapping

Objective: Observe where different variables live in memory and validate the OS memory layout.

A. C Source Code

```
pranisha@ubuntu:~/Desktop$ touch lab2_memory.c
pranisha@ubuntu:~/Desktop$ nano lab2_memory.c
```

The touch command was used to create an empty file named lab2_memory.c, and the nano editor was opened to write the source code.

```
  GNU nano 8.7.1                                    lab2_memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_var=150;
int bss_var;

int main() {
        int local_var=150;
        static int local_static=20;

        int *heap_var=(int *)malloc(sizeof(int));
        //Heap
        *heap_var=500;
        printf(".......Variable Addresses.......\n\n");
        printf("global_var:     %p (Data/BSS)\n", (void*)&global_var);
        printf("bss_var:        %p (Data/BSS)\n", (void*)&bss_var);
        printf("local_var:      %p (Stack)\n", (void*)&local_var);
        printf("local_static:   %p (Data/BSS)\n", (void*)&local_static);
        printf("heap_var ptr:   %p (Stack - pointer)\n", (void*)&heap_var);
        printf("heap_var:       %p (Heap - data)\n", (void*)heap_var);

        free(heap_var);
        return 0;
```

The program declares one variable for each memory segment: a global initialized variable (Data), a global uninitialized variable (BSS), a local variable (Stack) and a dynamically allocated variable created with malloc() (Heap). The address of each one is printed using %p.

B. Terminal Execution & Verification

```
pranisha@ubuntu:~/Desktop$ gcc lab2_memory.c -o lab2
pranisha@ubuntu:~/Desktop$ ./lab2
.......Variable Addresses.......

global_var:     0xab36f3760010 (Data/BSS)
bss_var:        0xab36f376001c (Data/BSS)
local_var:      0xffffd434b7ec (Stack)
local_static:   0xab36f3760014 (Data/BSS)
heap_var ptr:   0xffffd434b7f0 (Stack - pointer)
heap_var:       0xab3713376010 (Heap - data)
```

The program was compiled successfully and the output shows the address of each variable with its segment label (Data, BSS, Heap and Stack).

C. Highest and Lowest Addresses

The **Stack** variable has the highest address (starting with 0x7ff...). The **Data** and **BSS** variables have the lowest addresses of the four, and they sit right next to each other. The **Heap** address lies between the Data/BSS segment and the Stack.

D. Calculating the Stack and Heap Address Difference

Using the two addresses from your output, calculate the difference in the terminal:

```
pranisha@ubuntu:~/Desktop$ echo $((0xffffd434b7ec - 0xab3713376010))
93221208020956
```

The heap address was subtracted from the stack address using bash arithmetic. The result is a very large number of bytes, which shows that the OS keeps a wide gap between the stack and the heap.

E. Stack vs Heap Comparison

Data Type	Stack	Heap
Stores	Local variables, function calls	Dynamically allocated memory

		(malloc)
Managed by	Compiler and OS (automatic)	Programmer (malloc/free)
Address range	High (0x7ff…)	Lower, mid-range
Grows	Downward	Upward
Lifetime	Freed when the function returns	Stays until free() is called
Speed	Faster	Slower
Size	Small, fixed limit	Large, limited by available memory

The stack and heap sit at opposite ends of the process address space. The stack has the highest addresses and the Data and BSS segments have the lowest, with the heap in between. The stack and heap are separated by a very large gap, so they can grow toward each other (the stack downward, the heap upward) without colliding. The stack is managed automatically and freed when a function returns, while heap memory has to be freed manually using free().

III. Practical Code Example: Pointer and Memory Allocation:

A. C Source Code

```
pranisha@ubuntu:~/Desktop$ touch pointer_heap.c
pranisha@ubuntu:~/Desktop$ nano pointer_heap.c
```

The touch command was used to create an empty file named pointer_heap.c, and the nano editor was opened to write the source code.

```
 GNU nano 8.7.1                                    pointer_heap.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
        int *forheap;
        forheap=(int *)malloc(sizeof(int));
        *forheap=30;
        printf("Address of pointer (STACK):     %p\n", (void*)&forheap);
        printf("Address of data (HEAP):         %p\n", (void*)forheap);
        printf("Value stored:                   %d\n", *forheap);

        return 0;
}
```

The program declares a pointer forheap, which is a local variable and therefore lives on the stack. It then uses malloc() to allocate memory on the heap, stores the value 30 in it, prints both addresses, and finally releases the memory with free().

B. Terminal Execution & Verification

```
pranisha@ubuntu:~/Desktop$ gcc pointer_heap.c -o pointer_heap
pranisha@ubuntu:~/Desktop$ ./pointer_heap
Address of pointer (STACK):     0xffffc8da50f0
Address of data (HEAP):         0xc75b81ec9010
Value stored:                   30
```

The program was compiled without errors. The output shows the address of the pointer (Stack, starting with 0x7ff...), the address of the data it points to (Heap, a lower address) and the value stored, which is 30.

C. Observation

The pointer variable lives on the stack, but the memory it points to is allocated on the heap. This is why the two addresses are in different regions: the stack address is much higher than the heap address. The pointer bridges the two, since it is stored on the stack but holds the address of heap memory. The value is stored on the heap, so it has to be released manually with free().

IV. Summary and Key Takeaways

- Data types are contracts: they tell the OS how much memory to allocate and how to interpret it.
- Memory is segmented: the stack holds local variables, the heap holds dynamic memory, Data/BSS hold global variables and the text segment holds the code.
- The OS manages all of it: tracking, allocating, protecting and virtualizing memory.
- The stack is automatic while the heap is manual.
- Virtual memory provides isolation, so each process has its own address space.
- Pointers bridge the stack and the heap: the pointer variable lives on the stack but points to heap memory.

Knowledge Check Questions:

1. What is a data type in the context of programming and operating systems?

A data type is a contract between the programmer, compiler and OS. It defines how many bytes to allocate, how those bytes are interpreted, and which operations are valid on the data.

2. Name the five main memory segments in a process memory layout.

Text, Data, BSS, Heap and Stack.

3. What is the difference between the Data segment and the BSS segment?

The Data segment stores global and static variables that are initialized with a value, while the BSS segment stores those that are uninitialized. The OS sets BSS variables to zero when the program starts.

4. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

The stack starts at high addresses and grows downward, while the heap grows upward from lower addresses. This leaves a large free space between them so they can grow without overlapping. The text segment has a fixed size, so it sits at the bottom.

5. What happens to stack memory when a function returns? Who reclaims it?

When a function returns, its stack frame is removed and the stack pointer moves back, so the memory is freed automatically. This is handled by the compiler-generated code, and the programmer does not have to free it.

6. How does the OS know which memory belongs to which process?

Each process gets its own virtual address space. The OS keeps a separate page table for every process that maps virtual addresses to physical memory, so a process can only access the memory mapped to it.

V. Conclusion:

This lab showed how data types decide how much memory a variable uses and how the OS divides a process's memory into segments. Using sizeof() and %p, we observed the size of each data type and the addresses of variables in the Data, BSS, Heap and Stack segments. The results confirmed that the stack sits at the highest addresses, with the heap and the Data/BSS segments below it, and that a very large gap separates the stack and the heap.